

Bibliotecas JavaScript: jQuery

Domínio e Manipulação Avançada do Front-end

Prof.^a Catarina Silva

Ano Letivo 2025/2026 – 2º Semestre

Objetivos da Unidade

- Dominar a integração e configuração de bibliotecas externas.
- Compreender o paradigma de abstração do DOM através do jQuery.
- Aplicar seletores complexos e filtros de conteúdo.
- Implementar lógica de navegação na árvore DOM (Traversal).
- Desenvolver interfaces interativas com gestão avançada de eventos.
- Criar experiências de utilizador fluidas com animações e AJAX.

Agenda

Parte I: Fundamentos

- Bibliotecas vs. Frameworks
- Ecossistema jQuery
- Instalação e Preparação
- Objeto \$, noConflict e Ready

Parte II: O Motor de Seleção

- Seletores CSS e Hierárquicos
- Filtros e Atributos
- Navegação (Traversal)

Parte III: Manipulação e Eventos

- Manipulação de Conteúdo e DOM
- Gestão de Classes e Estilos
- Eventos e Delegação

Parte IV: UI e Comunicação

- Efeitos e Animações Custom
- Encadeamento (Chaining)
- AJAX e Promessas

Bibliotecas vs. Frameworks

■ Biblioteca (Library):

- Coleção de funções auxiliares.
- Decide quando chamar a biblioteca.
- Ex: jQuery, D3, Lodash.

■ Framework:

- Define a arquitetura da aplicação.
- *"O Framework está no controlo"* (Inversão de Controlo).
- Ex: Angular, Vue, React (embora React se auto-intitule biblioteca).

História

- Lançado em Janeiro de 2006 por John Resig.
- **Problema original:** Incompatibilidade massiva entre browsers (IE vs Firefox).
- **Filosofia:** *"Write Less, Do More"*.
- **Impacto:** Tornou-se a biblioteca JS mais usada do mundo (presente em >70% dos sites).
- **Legado:** Muitas das suas ideias foram integradas no padrão Web (querySelector, classList, fetch).

jQuery em 2026: Ainda é relevante?

- **Manutenção de Sistemas Legados:** Milhares de empresas usam jQuery em produção.
- **Rapidez de Prototipagem:** Plugins prontos a usar (Sliders, Modais, Datepickers).
- **WordPress:** O motor que alimenta 40% da web depende fortemente de jQuery.
- **Abstração Simples:** Às vezes o Vanilla JS é demasiado "verboso" para tarefas simples.
- **Cross-browser:** Garante comportamento idêntico mesmo em browsers mais restritos.

Instalação via CDN

- **CDN (Content Delivery Network):** Carrega a biblioteca de servidores geográficos distribuídos.

```
<!-- Google CDN -->  
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>  
  
<!-- jQuery Official CDN -->  
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
```

- **Vantagem:** Cache partilhada (se o utilizador já visitou outro site com jQuery, já o tem no PC).

Instalação Local e Versões

- **Versão de Produção (.min.js):** Comprimida, sem comentários, ideal para o site final.
- **Versão de Desenvolvimento (.js):** Legível, ideal para depuração (debug).
- **Ramos de Versão:**
 - 1.x: Suporte para browsers muito antigos (IE 6/7/8).
 - 2.x: Mais rápido, sem suporte para IE antigo.
 - 3.x: Versão atual, moderna, suporte a Promises, focada em performance.

O Objeto \$ e jQuery

- \$ é apenas um alias (atalho) para o objeto global jQuery.
- Quase tudo o que fazemos começa com \$().

```
$(document).ready(function() { ... });  
// Equivale a:  
jQuery(document).ready(function() { ... });  
  
// Podemos verificar a versao:  
console.log($.fn.jquery);
```

Gestão do Ciclo de Vida: Ready

- O código JS deve esperar que o HTML seja lido (*parsed*) antes de tentar seleccionar elementos.

```
// Forma classica
$(document).ready(function() {
    console.log("DOM pronto!");
});

// Forma moderna e curta
$(function() {
    console.log("DOM pronto!");
});
```

- **Diferença Crucial:** O ready dispara quando o HTML está pronto. O window.onload espera também por imagens e frames pesados.

Conflitos de Bibliotecas: noConflict()

- Outras bibliotecas (como Prototype) também usam o \$.

```
// Liberta o simbolo $
$.noConflict();

// Agora temos de usar jQuery
jQuery(document).ready(function() {
    jQuery("p").hide();
});

// Truque para continuar a usar $ internamente:
jQuery(document).ready(function($) {
    $("p").hide(); // Aqui o $ volta a funcionar
});
```

Seletores de Base (CSS)

```
$("*")           // Todos os elementos
$("div")         // Por tag
$("#topo")       // Por ID
$(".item")       // Por Classe
$(".item, h2")   // Multiplos seletores
```

- **Internamente:** O jQuery usa um motor chamado **Sizzle** para processar estes seletores de forma ultra-rápida.

Seletores Hierárquicos

```
$("#div p")           // Descendentes (qualquer nível)
$("#div > p")          // Filhos diretos
$("#label + input")    // Proximo irmao direto
$("#form ~ input")     // Todos os irmaos seguintes
```

Filtros de Posição

```
$("li:first")      // 0 primeiro li
$("li:last")       // 0 ultimo li
$("li:even")       // li's pares (0, 2, 4...)
$("li:odd")        // li's impares (1, 3, 5...)
$("li:eq(2)")      // li no indice 2 (terceiro)
$("li:gt(3)")      // li's com indice maior que 3
```

Filtros de Conteúdo e Estado

```
$( "div:contains('UA')" ) // Divs que contenham o texto 'UA'
$( "div:has(p)" )         // Divs que tenham um paragrafo dentro
$( "p:empty" )            // Paragrafos vazios

$( ":checked" )           // Checkboxes/Radios selecionados
$( ":selected" )          // Opcoes de Select selecionadas
$( ":enabled" )           // Elementos ativos
```

Seletores de Atributo

```
$("#[href]")           // Tem o atributo href
$("#[target='_blank']") // Valor exato
$("#[src$='.png']")      // Termina em .png
$("#[title*='ajuda']")  // Contem a palavra 'ajuda'
```


Navegar para Cima (Ancestrais)

- **parent()**: Sobe apenas um nível.
- **parents()**: Todos os ancestrais até ao html.
- **closest()**: O primeiro ancestral que coincida com o seletor (muito útil!).

```
$("#meuLink").parent().css("border", "1px solid red");  
$("#meuLink").closest(".container").hide();
```

Navegar para Baixo (Descendentes)

- **children()**: Filhos diretos.
- **find()**: Procura em todos os níveis abaixo (descendentes).

```
$("#ul").children(".ativo").show();  
$("#galeria").find("img").attr("title", "Imagem da Galeria");
```

Navegar Lateralmente (Irmãos)

- **next()** / **prev()**: Imediatamente a seguir/anterior.
- **nextAll()** / **prevAll()**: Todos os seguintes/anteriores.
- **siblings()**: Todos os irmãos (exceto o próprio).

```
$("h2").next("p").addClass("lead");
```

Manipular Conteúdo e Valores

```
// Ler valores
let html = $("#caixa").html();
let texto = $("#caixa").text();
let valor = $("#inputNome").val();

// Definir valores
$("#caixa").html("<p>Novo Conteudo</p>");
$("#caixa").text("Apenas texto puro");
$("#inputNome").val("Joao Silva");
```

Atributos e Propriedades

- **attr()**: Manipula atributos HTML (como aparecem no código).
- **prop()**: Manipula propriedades do objeto DOM (estado atual: checked, disabled).

```
$("#img").attr("src", "foto.jpg");  
$("#a").removeAttr("target");  
  
// Forma correta de marcar uma checkbox:  
$("#aceito").prop("checked", true);
```

Classes e Estilos Dinâmicos

```
$("#div").addClass("alerta erro");
$("#div").removeClass("erro");
$("#div").toggleClass("visivel"); // Inverte o estado

if ($("#div").hasClass("alerta")) {
    console.log("E um alerta!");
}

// CSS inline
$("#p").css("font-size", "18px");
$("#p").css({
    "color": "blue",
    "margin-top": "20px"
});
```

Modificar a Estrutura DOM

```
// Dentro do elemento selecionado
$("ul").append("<li>No fim</li>");
$("ul").prepend("<li>No inicio</li>");

// Fora (ao lado) do elemento selecionado
$("hr").after("<p>Linha de separacao</p>");
$("hr").before("<h3>Fim da Seccao</h3>");

// Envolver
$("img").wrap("<div class='moldura'></div>");
```

Remover e Esvaziar

- **remove()**: Remove o elemento e todos os seus dados/eventos.
- **empty()**: Remove apenas os filhos (conteúdo interno).
- **detach()**: Remove mas guarda na memória (útil para mover elementos sem perder eventos).

```
$("#lixo").remove();  
$("#lista").empty();
```


Eventos de Mouse e Teclado

```
// Mouse
$(".btn").click(function() { ... });
$(".btn").dblclick(function() { ... });
$(".caixa").mouseenter(function() { ... });

// Teclado
$(document).keydown(function(e) {
    console.log("Tecla pressionada: " + e.which);
});

// Formulario
$("#meuForm").submit(function(e) {
    e.preventDefault(); // Crucial!
});
```

O método on(): Delegação de Eventos

- Problema: `click()` não funciona em elementos criados dinamicamente no futuro.
- Solução: Delegação de eventos.

```
// Escutamos no PAI que ja existe, mas filtramos o FILHO
$("#lista").on("click", "li", function() {
    $(this).fadeOut();
});
```

- Desta forma, mesmo que adicione novos `li` via AJAX, eles já terão o evento de clique associado.

O Objeto Event

```
$("#a").click(function(event) {  
    event.preventDefault(); // Nao segue o link  
    event.stopPropagation(); // Nao sobe na hierarquia  
  
    console.log("Coordenada X: " + event.pageX);  
    console.log("Elemento clicado: " + event.target.nodeName);  
});
```

Efeitos de Visibilidade

- Podem receber duração ("slow", "fast" ou milissegundos).

```
$("#aviso").hide(1000);  
$("#aviso").show(500);  
$("#aviso").toggle(); // Alterna hide/show
```

Fade e Slide

```
// Opacidade (Fade)
$("#img").fadeIn();
$("#img").fadeOut();
$("#img").fadeTo("slow", 0.5); // Ate 50% de opacidade

// Altura (Slide)
$(".painel").slideDown();
$(".painel").slideUp();
$(".painel").slideToggle();
```

Animações Customizadas: animate()

- Permite animar propriedades numéricas.

```
$(".bola").animate({  
  left: '250px',  
  opacity: '0.5',  
  height: '150px',  
  width: '150px'  
}, 2000);
```

- *Nota: O jQuery base não anima cores. Para isso é necessário o jQuery UI.*

Chaining (Encadeamento)

- Cada método jQuery devolve o próprio objeto, permitindo "empilhar" comandos.

```
$("#meuElemento")  
  .css("color", "red")  
  .slideUp(1000)  
  .slideDown(1000)  
  .delay(2000)  
  .fadeOut();
```

- **Vantagem:** O browser não tem de procurar o elemento no DOM várias vezes. Performance superior!

O que é AJAX?

- **Asynchronous JavaScript And XML.**
- Troca de dados com o servidor em segundo plano.
- **Porquê usar jQuery para AJAX?**
 - O código nativo (XMLHttpRequest) era pesadíssimo.
 - O jQuery simplifica tudo para uma única função ou métodos curtos.

Métodos AJAX Curtos

```
// Pedido GET simples
$.get("api/usuarios", function(dados) {
    console.log(dados);
});

// Pedido POST com dados
$.post("api/guardar", { nome: "Ana", id: 10 }, function(res) {
    alert("Guardado: " + res.status);
});

// Carregar HTML diretamente num elemento
$("#conteudo").load("pagina_externa.html #seccao1");
```

Boas Práticas e Performance

- **Cache de Seletores:** Guarde seletores em variáveis se os usar mais do que uma vez.
 - Errado: `$(".item").hide(); $(".item").show();`
 - Certo: `let $item = $(".item"); $item.hide(); $item.show();`
- **Use IDs:** `$("id")` é muito mais rápido que `$(".class")`.
- **Minimize a manipulação do DOM:** Construa strings longas de HTML e faça um único `append()` no fim.
- **Vanilla vs jQuery:** Use jQuery quando a legibilidade e compatibilidade forem prioridade. Use Vanilla JS quando a performance absoluta for crítica.

jQuery vs. Vanilla JavaScript (Cheat Sheet)

Tarefa	jQuery	Vanilla JS (Moderno)
Seleção (ID)	<code>\$("#id")</code>	<code>document.getElementById("id")</code>
Seleção (Class)	<code>\$(".cl")</code>	<code>document.querySelectorAll(".cl")</code>
Conteúdo HTML	<code>el.html(«p>...»)</code>	<code>el.innerHTML = «p>...»</code>
Texto	<code>el.text("Olá")</code>	<code>el.textContent = "Olá"</code>
Estilo CSS	<code>el.css("color", "red")</code>	<code>el.style.color = "red"</code>
Adicionar Classe	<code>el.addClass("nova")</code>	<code>el.classList.add("nova")</code>
Remover Elemento	<code>el.remove()</code>	<code>el.remove()</code>
Evento Clique	<code>el.click(fn)</code>	<code>el.addEventListener("click", fn)</code>
AJAX (GET)	<code>\$.get(url, fn)</code>	<code>fetch(url).then(r => r.json())</code>

- **Vantagem jQuery:** Sintaxe curta, encadeamento, compatibilidade automática.
- **Vantagem Vanilla:** Performance nativa, zero dependências externas.

Comparação de Capacidades e Funcionalidades

Funcionalidade	jQuery	Vanilla JavaScript
Compatibilidade	Excelente (abstração automática de inconsistências)	Boa em browsers modernos; requer <i>polyfills</i> para antigos
Performance	Menor (devido à camada de abstração e processamento)	Superior (execução direta pelo motor do browser)
Peso / Carga	Adiciona 30KB comprimidos ao projeto	Zero overhead (já está no browser)
Efeitos	Métodos prontos (<i>slide, fade, toggle</i>)	Requer CSS Transitions ou Web Animations API
Ecossistema	Milhares de plugins prontos (Sliders, Grids)	Requer implementação manual ou módulos externos

Alpine.js: O "jQuery dos tempos modernos"

- **O que é?** Uma biblioteca minimalista para compor comportamento reativo diretamente no seu HTML.
- **Filosofia:** Inspirado no Vue, mas com a simplicidade do jQuery.
- **Vantagem:** Não requer compilação (*no build step*). Basta incluir uma tag `<script>`.
- **Ideal para:** Quando o jQuery parece "velho" e o React parece "demasiado pesado".

Sintaxe Declarativa vs Imperativa

jQuery (Imperativo)

```
$("#btn").click(function() {  
    $("#msg").toggle();  
});
```

"Dizemos ao browser PASSO A PASSO o que fazer."

Alpine.js (Declarativo)

```
<div x-data="{ open: false }">  
  <button @click="open = !open">  
    Toggle  
  </button>  
  <div x-show="open">01 !</div>  
</div>
```

"Declaramos o ESTADO e o HTML reage."

Diretivas Principais do Alpine

- **x-data:** Define um pedaço de estado (variáveis).
- **x-show / x-if:** Controla a visibilidade/presença do elemento.
- **x-on (ou @):** Escuta eventos (ex: @click, @submit).
- **x-bind (ou :):** Liga atributos HTML a variáveis do JS.
- **x-model:** Ligação bidirecional (*two-way binding*) para formulários.

Porquê escolher Alpine?

- **Peso Pluma:** Apenas 15KB (metade do jQuery).
- **Fácil Aprendizagem:** Se sabe HTML e um pouco de JS, aprende em 15 minutos.
- **Código Localizado:** A lógica está junto ao elemento HTML que ela controla.
- **Performance:** Muito mais rápido que o jQuery em manipulações de estado.

Cenários de Uso: Alpine.js

- Dashboards simples.
- Landing pages com componentes interativos (modais, carrosséis).
- Pequenos formulários com validação em tempo real.
- Substituição direta de pequenos scripts jQuery em projetos antigos.

A Era dos Frameworks Reativos

- O foco mudou de "Manipular Elementos" para "Gerir Dados".
- **Os Três Pilares:**
 - **React:** Focado em componentes e flexibilidade (Meta).
 - **Vue.js:** Equilíbrio entre simplicidade e poder.
 - **Svelte:** Compila o código para Vanilla JS puro (Performance máxima).
- **Diferença Fundamental:** Virtual DOM e Reatividade granular.

Vantagens e Desvantagens

Vantagens

- Escalabilidade para apps gigantes.
- Reutilização de componentes.
- Ecossistemas massivos (Bibliotecas UI, Roteamento).
- Facilidade de testes unitários.

Desvantagens

- Curva de aprendizagem acentuada.
- Requerem ferramentas de *build* (Node.js, Vite, Webpack).
- "Overhead" para sites simples.
- SEO requer cuidados extras (SSR/SSG).

Custos: Desenvolvimento e Manutenção

- **Configuração:** Requer setup de ambiente (NPM, compiladores).
- **Atualizações:** Frameworks evoluem rápido (risco de *breaking changes*).
- **Especialização:** Programadores React/Vue são mais caros no mercado.
- **Performance de Carregamento:** O browser tem de descarregar o framework antes de mostrar a página.

Qual usar? (A Regra de Ouro)

- **Use Vanilla JS:** Se o projeto é ultra-simples e a performance é crítica.
- **Use jQuery / Alpine:** Se é um site institucional, CMS (WordPress) ou precisa de interatividade rápida.
- **Use React / Vue:** Se está a construir uma aplicação (ex: Facebook, Gmail, Spotify) onde o utilizador interage com dados o tempo todo.
- **Use Svelte:** Se quer a experiência de um framework mas a velocidade do Vanilla JS.

Resumo de Escolha por Cenário

Tipo de Projeto	Complexidade	Tecnologia Recomendada
Blog / Site Simples	Baixa	Vanilla JS / Alpine.js
E-Commerce Tradicional	Média	jQuery / Alpine.js
Dashboard de Dados	Alta	Vue.js / React
App Mobile / Web Complexa	Muito Alta	React / Svelte

Onde continuar a aprender?

- **W3Schools:**

- jQuery Tutorial
- JavaScript Tutorial

- **Cheat Sheets (Folhas de Consulta):**

- jQuery Cheat Sheet (Interativa)
- Alpine.js Documentation
- JavaScript Cheat Sheet

- **Documentação Oficial:**

- jQuery API Documentation
- MDN Web Docs (JavaScript Nativo)

Dúvidas?

Próximo passo: Guião Prático e Resoluções.